

Complete Banking Analytics Project

Project Overview

You have a comprehensive banking dataset with 10 CSV files:

- **customers.csv** (1000 rows)
- **accounts.csv** (1400 rows)
- **transactions.csv** (10,000 rows)
- **loans.csv** (800 rows)
- **credit_cards.csv** (900 rows)
- **branches.csv** (50 rows)
- **employees.csv** (500 rows)
- **fraud_transactions.csv** (600 rows)
- **insurance_products.csv** (700 rows)
- **customer_support_tickets.csv** (1200 rows)

Phase 1: Azure Infrastructure Setup

Step 1.1: Create Resource Group

GUI Steps:

1. Go to [Azure Portal](#)
2. Click **"Create a resource"** → Search **"Resource group"**
3. Fill in:
 - **Subscription:** Your subscription
 - **Resource group name:** rg-banking-analytics-prod
 - **Region:** Central US
4. Click **"Review + create"** → **"Create"**

Step 1.2: Create Storage Account (ADLS Gen2)

GUI Steps:

1. Click **"Create a resource"** → Search **"Storage account"**
2. Fill in:
 - **Subscription:** Your subscription
 - **Resource group:** rg-banking-analytics-prod
 - **Storage account name:** bankingdatalake789 (must be unique)
 - **Region:** Central US
 - **Performance:** Standard
 - **Redundancy:** Locally-redundant storage (LRS)
3. Click **"Advanced"** tab:
 - **Enable hierarchical namespace:** Checked ✓
4. Click **"Review + create"** → **"Create"**

Step 1.3: Create Containers

GUI Steps:

1. Go to your storage account → **"Containers"** (left menu)
2. Click **"+ Container"** for each:
 - **Name:** raw → **Public access level:** Private → **Create**
 - **Name:** bronze → **Public access level:** Private → **Create**
 - **Name:** silver → **Public access level:** Private → **Create**
 - **Name:** gold → **Public access level:** Private → **Create**
 - **Name:** archive → **Public access level:** Private → **Create**

Alternative using Cloud Shell:

```
bash
storageAccountName="bankingdatalake789"

for container in raw bronze silver gold archive
do
az storage container create \
    --name $container \
    --account-name $storageAccountName \
    --auth-mode login
done
```

Step 1.4: Create Databricks Workspace

GUI Steps:

1. Click **"Create a resource"** → Search **"Azure Databricks"**
2. Fill in:
 - **Workspace name:** db-banking-analytics-prod
 - **Resource group:** rg-banking-analytics-prod
 - **Location:** Central US
 - **Pricing Tier:** Premium (required for Unity Catalog)
3. Click **"Review + create"** → **"Create"**

Step 1.5: Create Synapse Analytics Workspace

GUI Steps:

1. Click **"Create a resource"** → Search **"Azure Synapse Analytics"**
2. Fill in:
 - **Workspace name:** synapse-banking-prod
 - **Resource group:** rg-banking-analytics-prod
 - **Region:** Central US
 - **Data Lake Storage Gen2:** Select your storage account bankingdatalake789
 - **File system:** gold
3. Click **"Review + create"** → **"Create"**

Phase 2: Upload Datasets to Raw Container

Step 2.1: Upload Files Using GUI

GUI Steps:

1. Go to storage account bankingdatalake789
2. Click **"Containers"** → Click **"raw"**
3. Click **"Upload"** → **"Browse for files"**
4. Select all 10 CSV files:
 - customers.csv
 - accounts.csv
 - transactions.csv
 - loans.csv
 - credit_cards.csv
 - branches.csv
 - employees.csv
 - fraud_transactions.csv
 - insurance_products.csv
 - customer_support_tickets.csv
5. Click **"Upload"**

Step 2.2: Create Folder Structure

Create subfolders for better organization:

- . raw/banking/customers/
- . raw/banking/accounts/
- . raw/banking/transactions/
- . etc.

Phase 3: Databricks Unity Catalog Setup

Step 3.1: Launch Databricks Workspace

GUI Steps:

1. Go to Azure Portal → db-banking-analytics-prod
2. Click "Launch Workspace"

Step 3.2: Create Access Connector for Databricks

GUI Steps:

1. In Azure Portal → "Create a resource" → Search "Access Connector for Databricks"
2. Fill in:
 - **Connector name:** adls-access-connector-banking
 - **Resource group:** rg-banking-analytics-prod
 - **Region:** Central US
3. Click "Create"

Step 3.3: Grant RBAC Permissions to Access Connector

GUI Steps:

1. Go to storage account bankingdatalake789
2. Click "Access Control (IAM)" → "+ Add" → "Add role assignment"
3. Select role: "Storage Blob Data Contributor"
4. Assign access to: "Managed identity"
5. Select: "Access Connector for Databricks" → Choose adls-access-connector-banking
6. Click "Review + assign"

Step 3.4: Create Databricks Cluster

GUI Steps in Databricks Workspace:

1. Click "Compute" (left sidebar)
2. Click "Create Cluster"
3. Configure:
 - **Cluster name:** banking-cluster-prod
 - **Runtime:** 13.3 LTS (Scala 2.12, Spark 3.4.1)
 - **Unity Catalog:** Enabled (toggle ON)
 - **Node type:** Standard_DS3_v2
 - **Min workers:** 2
 - **Max workers:** 8
4. Click "Create Cluster"

Step 3.5: Create Storage Credential

SQL in Databricks Notebook:

```
sql
-- Create storage credential for ADLS access
CREATE STORAGE CREDENTIAL adls_credential_banking
WITH (
    AZURE_MI = '/subscriptions/YOUR_SUBSCRIPTION_ID/resourceGroups/rg-banking-analytics-
prod/providers/Microsoft.Databricks/accessConnectors/adls-access-connector-banking'
);
```



```
-- Verify credential
DESCRIBE STORAGE CREDENTIAL adls_credential_banking;
```

Step 3.6: Create External Locations

```
sql
-- Raw container Location
CREATE EXTERNAL LOCATION raw_location_banking
URL 'abfss://raw@bankingdatalake789.dfs.core.windows.net/'
WITH (STORAGE CREDENTIAL adls_credential_banking);

-- Bronze container Location
CREATE EXTERNAL LOCATION bronze_location_banking
URL 'abfss://bronze@bankingdatalake789.dfs.core.windows.net/'
WITH (STORAGE CREDENTIAL adls_credential_banking);

-- Silver container Location
CREATE EXTERNAL LOCATION silver_location_banking
URL 'abfss://silver@bankingdatalake789.dfs.core.windows.net/'
WITH (STORAGE CREDENTIAL adsl_credential_banking);

-- Gold container Location
CREATE EXTERNAL LOCATION gold_location_banking
URL 'abfss://gold@bankingdatalake789.dfs.core.windows.net/'
WITH (STORAGE CREDENTIAL adls_credential_banking);

-- Show all external locations
SHOW EXTERNAL LOCATIONS;
```

Step 3.7: Create Catalog and Schemas

```
sql
-- Create banking catalog
CREATE CATALOG banking_catalog
COMMENT "Banking analytics catalog"
MANAGED LOCATION 'abfss://bronze@bankingdatalake789.dfs.core.windows.net/';

-- Create schemas for medallion layers
CREATE SCHEMA banking_catalog.bronze
COMMENT "Raw banking data"
MANAGED LOCATION 'abfss://bronze@bankingdatalake789.dfs.core.windows.net/';

CREATE SCHEMA banking_catalog.silver
COMMENT "Cleaned banking data"
MANAGED LOCATION 'abfss://silver@bankingdatalake789.dfs.core.windows.net/';

CREATE SCHEMA banking_catalog.gold
COMMENT "Aggregated banking analytics"
MANAGED LOCATION 'abfss://gold@bankingdatalake789.dfs.core.windows.net/';

-- Show catalogs
SHOW CATALOGS;

-- Show schemas
SHOW SCHEMAS IN banking_catalog;
```

Phase 4: Read All Datasets from Raw Container

Notebook: 00_read_all_datasets


```

python
# Databricks Notebook: 00_read_all_datasets
# Purpose: Read all 10 banking CSV files from ADLS Raw container

print("="*60)
print("READING ALL BANKING DATASETS FROM ADLS")
print("="*60)

# Storage account configuration
storage_account = "bankingdatalake789"
raw_base_path = f"abfss://raw@{storage_account}.dfs.core.windows.net/banking/"

# =====
# 1. Read Customers Dataset
# =====
print("\n1. Reading Customers Dataset...")
customers_path = f"{raw_base_path}customers.csv"
df_customers = spark.read.option("header", True).option("inferSchema", True).csv(customers_path)
print(f"✓ Customers records: {df_customers.count()}")
print("Schema:")
df_customers.printSchema()
display(df_customers.limit(5))

# =====
# 2. Read Accounts Dataset
# =====
print("\n2. Reading Accounts Dataset...")
accounts_path = f"{raw_base_path}accounts.csv"
df_accounts = spark.read.option("header", True).option("inferSchema", True).csv(accounts_path)
print(f"✓ Accounts records: {df_accounts.count()}")
display(df_accounts.limit(5))

# =====
# 3. Read Transactions Dataset
# =====
print("\n3. Reading Transactions Dataset...")
transactions_path = f"{raw_base_path}transactions.csv"
df_transactions = spark.read.option("header", True).option("inferSchema",
True).csv(transactions_path)
print(f"✓ Transactions records: {df_transactions.count()}")
display(df_transactions.limit(5))

# =====
# 4. Read Loans Dataset
# =====
print("\n4. Reading Loans Dataset...")
loans_path = f"{raw_base_path}loans.csv"
df_loans = spark.read.option("header", True).option("inferSchema", True).csv(loans_path)
print(f"✓ Loans records: {df_loans.count()}")
display(df_loans.limit(5))

# =====
# 5. Read Credit Cards Dataset
# =====
print("\n5. Reading Credit Cards Dataset...")
cards_path = f"{raw_base_path}credit_cards.csv"
df_cards = spark.read.option("header", True).option("inferSchema", True).csv(cards_path)
print(f"✓ Credit Cards records: {df_cards.count()}")
display(df_cards.limit(5))

```

```

# =====
# 6. Read Branches Dataset
# =====
print("\n6. Reading Branches Dataset...")
branches_path = f"{raw_base_path}branches.csv"
df_branches = spark.read.option("header", True).option("inferSchema", True).csv(branches_path)
print(f"✓ Branches records: {df_branches.count()}")
display(df_branches.limit(5))

# =====
# 7. Read Employees Dataset
# =====
print("\n7. Reading Employees Dataset...")
employees_path = f"{raw_base_path}employees.csv"
df_employees = spark.read.option("header", True).option("inferSchema", True).csv(employees_path)
print(f"✓ Employees records: {df_employees.count()}")
display(df_employees.limit(5))

# =====
# 8. Read Fraud Transactions Dataset
# =====
print("\n8. Reading Fraud Transactions Dataset...")
fraud_path = f"{raw_base_path}fraud_transactions.csv"
df_fraud = spark.read.option("header", True).option("inferSchema", True).csv(fraud_path)
print(f"✓ Fraud Transactions records: {df_fraud.count()}")
display(df_fraud.limit(5))

# =====
# 9. Read Insurance Products Dataset
# =====
print("\n9. Reading Insurance Products Dataset...")
insurance_path = f"{raw_base_path}insurance_products.csv"
df_insurance = spark.read.option("header", True).option("inferSchema", True).csv(insurance_path)
print(f"✓ Insurance Products records: {df_insurance.count()}")
display(df_insurance.limit(5))

# =====
# 10. Read Customer Support Tickets Dataset
# =====
print("\n10. Reading Customer Support Tickets Dataset...")
tickets_path = f"{raw_base_path}customer_support_tickets.csv"
df_tickets = spark.read.option("header", True).option("inferSchema", True).csv(tickets_path)
print(f"✓ Support Tickets records: {df_tickets.count()}")
display(df_tickets.limit(5))

# =====
# Summary Report
# =====
print("\n" + "="*60)
print("DATASET SUMMARY REPORT")
print("="*60)

summary_data = [
    ("Customers", df_customers.count(), len(df_customers.columns)),
    ("Accounts", df_accounts.count(), len(df_accounts.columns)),
    ("Transactions", df_transactions.count(), len(df_transactions.columns)),
    ("Loans", df_loans.count(), len(df_loans.columns)),
    ("Credit Cards", df_cards.count(), len(df_cards.columns)),
    ("Branches", df_branches.count(), len(df_branches.columns)),
    ("Employees", df_employees.count(), len(df_employees.columns)),

```

```

    ("Fraud Transactions", df_fraud.count(), len(df_fraud.columns)),
    ("Insurance Products", df_insurance.count(), len(df_insurance.columns)),
    ("Support Tickets", df_tickets.count(), len(df_tickets.columns))
]

from pyspark.sql import Row
summary_df = spark.createDataFrame(
    [Row(Table=row[0], Record_Count=row[1], Columns=row[2]) for row in summary_data]
)
display(summary_df)

print("\n All 10 datasets loaded successfully!")

```

Phase 5: Read with Explicit Schemas (Better Performance)

Notebook: 01_read_with_schemas

```

python
# Databricks Notebook: 01_read_with_schemas
# Purpose: Read all datasets with explicit schemas for better performance

from pyspark.sql.functions import *
from pyspark.sql.types import *
from datetime import datetime

print("="*60)
print("READING BANKING DATASETS WITH EXPLICIT SCHEMAS")
print("="*60)

storage_account = "bankingdatalake789"
raw_base_path = f"abfss://raw@{storage_account}.dfs.core.windows.net/banking/"

# =====
# Define schemas for all tables
# =====

customers_schema = StructType([
    StructField("customer_id", StringType(), True),
    StructField("first_name", StringType(), True),
    StructField("last_name", StringType(), True),
    StructField("dob", DateType(), True),
    StructField("gender", StringType(), True),
    StructField("phone", StringType(), True),
    StructField("email", StringType(), True),
    StructField("city", StringType(), True),
    StructField("state", StringType(), True),
    StructField("customer_since", DateType(), True),
    StructField("kyc_status", StringType(), True),
    StructField("occupation", StringType(), True),
    StructField("annual_income", IntegerType(), True),
    StructField("risk_category", StringType(), True)
])

accounts_schema = StructType([
    StructField("account_id", StringType(), True),
    StructField("customer_id", StringType(), True),
    StructField("branch_id", StringType(), True),
    StructField("account_type", StringType(), True),
    StructField("account_number", StringType(), True),

```



```

    StructField("open_date", DateType(), True),
    StructField("balance", DoubleType(), True),
    StructField("status", StringType(), True),
    StructField("currency", StringType(), True),
    StructField("nominee_registered", StringType(), True)
])

transactions_schema = StructType([
    StructField("transaction_id", StringType(), True),
    StructField("account_id", StringType(), True),
    StructField("transaction_date", DateType(), True),
    StructField("transaction_type", StringType(), True),
    StructField("amount", DoubleType(), True),
    StructField("merchant_name", StringType(), True),
    StructField("payment_mode", StringType(), True),
    StructField("status", StringType(), True),
    StructField("city", StringType(), True),
    StructField("channel", StringType(), True)
])

loans_schema = StructType([
    StructField("loan_id", StringType(), True),
    StructField("customer_id", StringType(), True),
    StructField("loan_type", StringType(), True),
    StructField("loan_amount", DoubleType(), True),
    StructField("interest_rate", DoubleType(), True),
    StructField("tenure_months", IntegerType(), True),
    StructField("emi_amount", DoubleType(), True),
    StructField("loan_start_date", DateType(), True),
    StructField("loan_status", StringType(), True),
    StructField("collateral_required", StringType(), True)
])

credit_cards_schema = StructType([
    StructField("card_id", StringType(), True),
    StructField("customer_id", StringType(), True),
    StructField("card_type", StringType(), True),
    StructField("credit_limit", IntegerType(), True),
    StructField("available_limit", DoubleType(), True),
    StructField("outstanding_balance", DoubleType(), True),
    StructField("issue_date", DateType(), True),
    StructField("expiry_date", DateType(), True),
    StructField("card_status", StringType(), True)
])

branches_schema = StructType([
    StructField("branch_id", StringType(), True),
    StructField("branch_name", StringType(), True),
    StructField("city", StringType(), True),
    StructField("state", StringType(), True),
    StructField("ifsc_code", StringType(), True),
    StructField("manager_name", StringType(), True),
    StructField("open_date", DateType(), True)
])

employees_schema = StructType([
    StructField("employee_id", StringType(), True),
    StructField("branch_id", StringType(), True),
    StructField("employee_name", StringType(), True),
    StructField("designation", StringType(), True),
    StructField("salary", IntegerType(), True),

```



```

        StructField("joining_date", DateType(), True),
        StructField("employment_status", StringType(), True)
    ])

fraud_schema = StructType([
    StructField("fraud_id", StringType(), True),
    StructField("transaction_id", StringType(), True),
    StructField("fraud_type", StringType(), True),
    StructField("risk_score", IntegerType(), True),
    StructField("detected_date", DateType(), True),
    StructField("investigation_status", StringType(), True),
    StructField("loss_amount", DoubleType(), True)
])

insurance_schema = StructType([
    StructField("policy_id", StringType(), True),
    StructField("customer_id", StringType(), True),
    StructField("policy_type", StringType(), True),
    StructField("premium_amount", IntegerType(), True),
    StructField("sum_assured", IntegerType(), True),
    StructField("start_date", DateType(), True),
    StructField("end_date", DateType(), True),
    StructField("status", StringType(), True)
])

tickets_schema = StructType([
    StructField("ticket_id", StringType(), True),
    StructField("customer_id", StringType(), True),
    StructField("issue_type", StringType(), True),
    StructField("priority", StringType(), True),
    StructField("status", StringType(), True),
    StructField("created_date", DateType(), True),
    StructField("resolved_date", DateType(), True),
    StructField("channel", StringType(), True)
])

# =====
# Read all datasets with schemas
# =====

print("\nReading datasets with explicit schemas...\n")

# Customers
df_customers = spark.read.option("header",
True).schema(customers_schema).csv(f"{raw_base_path}customers.csv")
print(f"✓ Customers: {df_customers.count()} records")

# Accounts
df_accounts = spark.read.option("header",
True).schema(accounts_schema).csv(f"{raw_base_path}accounts.csv")
print(f"✓ Accounts: {df_accounts.count()} records")

# Transactions
df_transactions = spark.read.option("header",
True).schema(transactions_schema).csv(f"{raw_base_path}transactions.csv")
print(f"✓ Transactions: {df_transactions.count()} records")

# Loans
df_loans = spark.read.option("header", True).schema(loans_schema).csv(f"{raw_base_path}loans.csv")
print(f"✓ Loans: {df_loans.count()} records")

```

```

# Credit Cards
df_cards = spark.read.option("header",
True).schema(credit_cards_schema).csv(f"{raw_base_path}credit_cards.csv")
print(f"✓ Credit Cards: {df_cards.count()} records")

# Branches
df_branches = spark.read.option("header",
True).schema(branches_schema).csv(f"{raw_base_path}branches.csv")
print(f"✓ Branches: {df_branches.count()} records")

# Employees
df_employees = spark.read.option("header",
True).schema(employees_schema).csv(f"{raw_base_path}employees.csv")
print(f"✓ Employees: {df_employees.count()} records")

# Fraud
df_fraud = spark.read.option("header",
True).schema(fraud_schema).csv(f"{raw_base_path}fraud_transactions.csv")
print(f"✓ Fraud: {df_fraud.count()} records")

# Insurance
df_insurance = spark.read.option("header",
True).schema(insurance_schema).csv(f"{raw_base_path}insurance_products.csv")
print(f"✓ Insurance: {df_insurance.count()} records")

# Support Tickets
df_tickets = spark.read.option("header",
True).schema(tickets_schema).csv(f"{raw_base_path}customer_support_tickets.csv")
print(f"✓ Support Tickets: {df_tickets.count()} records")

print("\n All 10 datasets loaded successfully with explicit schemas!")

```

Phase 6: Create Bronze Delta Tables

Notebook: 02_create_bronze_delta_tables

python

Databricks Notebook: 02_create_bronze_delta_tables

Purpose: Write all banking datasets as Delta tables to Bronze container

```

from pyspark.sql.functions import input_file_name, current_timestamp, lit
from pyspark.sql.types import *
from datetime import datetime

```

```

print("="*60)
print("CREATING BRONZE DELTA TABLES FOR ALL DATASETS")
print("="*60)

```

```

storage_account = "bankingdatalake789"
raw_base_path = f"abfss://raw@{storage_account}.dfs.core.windows.net/banking/"
bronze_base_path = f"abfss://bronze@{storage_account}.dfs.core.windows.net/"

```

```

# =====
# Define schemas (same as above - omitted for brevity)
# =====

```

Read and Write each table as Delta

1. Customers Table

```
print("Processing Customers...")
df_customers = spark.read.option("header",
True).schema(customers_schema).csv(f"{raw_base_path}customers.csv")
df_customers_with_metadata = df_customers \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("customers.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_customers_path = f"{bronze_base_path}customers/"
df_customers_with_metadata.write.format("delta").mode("overwrite").save(bronze_customers_path)
print(f"✓ Customers Delta table created")
```

2. Accounts Table

```
print("Processing Accounts...")
df_accounts = spark.read.option("header",
True).schema(accounts_schema).csv(f"{raw_base_path}accounts.csv")
df_accounts_with_metadata = df_accounts \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("accounts.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_accounts_path = f"{bronze_base_path}accounts/"
df_accounts_with_metadata.write.format("delta").mode("overwrite").save(bronze_accounts_path)
print(f"✓ Accounts Delta table created")
```

3. Transactions Table

```
print("Processing Transactions...")
df_transactions = spark.read.option("header",
True).schema(transactions_schema).csv(f"{raw_base_path}transactions.csv")
df_transactions_with_metadata = df_transactions \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("transactions.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_transactions_path = f"{bronze_base_path}transactions/"
df_transactions_with_metadata.write.format("delta").mode("overwrite").save(bronze_transactions_path)
print(f"✓ Transactions Delta table created")
```

4. Loans Table

```
print("Processing Loans...")
df_loans = spark.read.option("header", True).schema(loans_schema).csv(f"{raw_base_path}loans.csv")
df_loans_with_metadata = df_loans \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("loans.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_loans_path = f"{bronze_base_path}loans/"
df_loans_with_metadata.write.format("delta").mode("overwrite").save(bronze_loans_path)
print(f"✓ Loans Delta table created")
```

5. Credit Cards Table

```
print("Processing Credit Cards...")
df_cards = spark.read.option("header",
True).schema(credit_cards_schema).csv(f"{raw_base_path}credit_cards.csv")
df_cards_with_metadata = df_cards \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("credit_cards.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_cards_path = f"{bronze_base_path}credit_cards/"
df_cards_with_metadata.write.format("delta").mode("overwrite").save(bronze_cards_path)
print(f"✓ Credit Cards Delta table created")
```


6. Branches Table

```
print("Processing Branches...")
df_branches = spark.read.option("header",
True).schema(branches_schema).csv(f"{raw_base_path}branches.csv")
df_branches_with_metadata = df_branches \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("branches.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_branches_path = f"{bronze_base_path}branches/"
df_branches_with_metadata.write.format("delta").mode("overwrite").save(bronze_branches_path)
print(f"✓ Branches Delta table created")
```

7. Employees Table

```
print("Processing Employees...")
df_employees = spark.read.option("header",
True).schema(employees_schema).csv(f"{raw_base_path}employees.csv")
df_employees_with_metadata = df_employees \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("employees.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_employees_path = f"{bronze_base_path}employees/"
df_employees_with_metadata.write.format("delta").mode("overwrite").save(bronze_employees_path)
print(f"✓ Employees Delta table created")
```

8. Fraud Table

```
print("Processing Fraud...")
df_fraud = spark.read.option("header",
True).schema(fraud_schema).csv(f"{raw_base_path}fraud_transactions.csv")
df_fraud_with_metadata = df_fraud \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("fraud_transactions.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_fraud_path = f"{bronze_base_path}fraud_transactions/"
df_fraud_with_metadata.write.format("delta").mode("overwrite").save(bronze_fraud_path)
print(f"✓ Fraud Delta table created")
```

9. Insurance Table

```
print("Processing Insurance...")
df_insurance = spark.read.option("header",
True).schema(insurance_schema).csv(f"{raw_base_path}insurance_products.csv")
df_insurance_with_metadata = df_insurance \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("insurance_products.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_insurance_path = f"{bronze_base_path}insurance_products/"
df_insurance_with_metadata.write.format("delta").mode("overwrite").save(bronze_insurance_path)
print(f"✓ Insurance Delta table created")
```

10. Support Tickets Table

```
print("Processing Support Tickets...")
df_tickets = spark.read.option("header",
True).schema(tickets_schema).csv(f"{raw_base_path}customer_support_tickets.csv")
df_tickets_with_metadata = df_tickets \
    .withColumn("ingestion_timestamp", current_timestamp()) \
    .withColumn("source_file", lit("customer_support_tickets.csv")) \
    .withColumn("ingestion_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
bronze_tickets_path = f"{bronze_base_path}customer_support_tickets/"
df_tickets_with_metadata.write.format("delta").mode("overwrite").save(bronze_tickets_path)
print(f"✓ Support Tickets Delta table created")
```



```
print("\n  ALL BRONZE DELTA TABLES CREATED SUCCESSFULLY!")
```

Phase 7: Silver Layer Transformations

Notebook: 03_silver_transformations

```
python
```

```
# Databricks Notebook: 03_silver_transformations
```

```
# Purpose: Transform Bronze data to Silver Layer
```

```
from pyspark.sql.functions import *
from pyspark.sql.types import *
from pyspark.sql.window import Window
from datetime import datetime
```

```
print("="*70)
print("SILVER LAYER TRANSFORMATIONS - BANKING DATASETS")
print("="*70)
```

```
storage_account = "bankingdatalake789"
bronze_base_path = f"abfss://bronze@{storage_account}.dfs.core.windows.net/"
silver_base_path = f"abfss://silver@{storage_account}.dfs.core.windows.net/"
```

```
# =====
```

```
# 1. CUSTOMERS TABLE - Silver Layer
```

```
# =====
```

```
print("\n1.1 Processing Customers Data (Bronze → Silver)...")
```

```
bronze_customers_path = f"{bronze_base_path}customers/"
df_customers_bronze = spark.read.format("delta").load(bronze_customers_path)
print(f"  Bronze records: {df_customers_bronze.count()}")
```

```
df_customers_silver = df_customers_bronze \
    .dropDuplicates(["customer_id"]) \
    .filter(col("customer_id").isNotNull()) \
    .withColumn("age", floor(datediff(current_date(), col("dob")) / 365.25)) \
    .withColumn("age_group",
        when(col("age") < 25, "Young (<25)")
        .when(col("age") < 40, "Adult (25-39)")
        .when(col("age") < 60, "Middle Age (40-59)")
        .otherwise("Senior (60+)")) \
    .withColumn("full_name", concat(initcap(col("first_name")), lit(" "),
initcap(col("last_name")))) \
    .withColumn("gender_standardized",
        when(upper(col("gender")).isin("M", "MALE"), "Male")
        .when(upper(col("gender")).isin("F", "FEMALE"), "Female")
        .otherwise("Other")) \
    .withColumn("income_bracket",
        when(col("annual_income") < 500000, "Low (<5L)")
        .when(col("annual_income") < 1500000, "Medium (5L-15L)")
        .when(col("annual_income") < 3000000, "High (15L-30L)")
        .otherwise("Very High (>30L))) \
    .withColumn("email_domain", regexp_extract(col("email"), "@(.+)", 1)) \
    .withColumn("customer_tenure_years", floor(datediff(current_date(), col("customer_since")) /
365.25)) \
    .withColumn("is_kyc_verified", col("kyc_status") == "Verified") \
    .withColumn("silver_processing_timestamp", current_timestamp()) \
    .withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))
```

```

        .drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f"  Silver records: {df_customers_silver.count()}")
display(df_customers_silver.limit(3))

silver_customers_path = f"{silver_base_path}customers_silver/"
df_customers_silver.write.format("delta").mode("overwrite").option("overwriteSchema",
"true").save(silver_customers_path)
print(f"  ✓ Saved to: {silver_customers_path}")

# =====
# 2. ACCOUNTS TABLE - Silver Layer
# =====
print("\n1.2 Processing Accounts Data (Bronze → Silver)...")

bronze_accounts_path = f"{bronze_base_path}accounts/"
df_accounts_bronze = spark.read.format("delta").load(bronze_accounts_path)
print(f"  Bronze records: {df_accounts_bronze.count()}")

df_accounts_silver = df_accounts_bronze \
    .dropDuplicates(["account_id"]) \
    .filter(col("account_id").isNotNull()) \
    .withColumn("account_status_standardized",
        when(upper(col("status")).isin("ACTIVE"), "Active")
        .when(upper(col("status")).isin("INACTIVE"), "Inactive")
        .when(upper(col("status")).isin("FROZEN"), "Frozen")
        .when(upper(col("status")).isin("CLOSED"), "Closed")
        .otherwise("Unknown")) \
    .withColumn("is_active", col("account_status_standardized") == "Active") \
    .withColumn("balance_category",
        when(col("balance") < 50000, "Low (<50K)")
        .when(col("balance") < 200000, "Medium (50K-200K)")
        .when(col("balance") < 500000, "High (200K-500K)")
        .otherwise("Very High (>500K))) \
    .withColumn("has_nominee", col("nominee_registered") == "Yes") \
    .withColumn("account_age_days", datediff(current_date(), col("open_date"))) \
    .withColumn("account_age_years", floor(col("account_age_days") / 365.25)) \
    .withColumn("silver_processing_timestamp", current_timestamp()) \
    .withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S"))) \
    .drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f"  Silver records: {df_accounts_silver.count()}")
display(df_accounts_silver.limit(3))

silver_accounts_path = f"{silver_base_path}accounts_silver/"
df_accounts_silver.write.format("delta").mode("overwrite").option("overwriteSchema",
"true").save(silver_accounts_path)
print(f"  ✓ Saved to: {silver_accounts_path}")

# =====
# 3. TRANSACTIONS TABLE - Silver Layer
# =====
print("\n1.3 Processing Transactions Data (Bronze → Silver)...")

bronze_transactions_path = f"{bronze_base_path}transactions/"
df_transactions_bronze = spark.read.format("delta").load(bronze_transactions_path)
print(f"  Bronze records: {df_transactions_bronze.count()}")

df_transactions_silver = df_transactions_bronze \
    .dropDuplicates(["transaction_id"]) \
    .filter(col("transaction_id").isNotNull()) \

```

```

.withColumn("transaction_year", year(col("transaction_date"))) \
.withColumn("transaction_month", month(col("transaction_date"))) \
.withColumn("transaction_day", dayofmonth(col("transaction_date"))) \
.withColumn("transaction_quarter", quarter(col("transaction_date"))) \
.withColumn("transaction_week", weekofyear(col("transaction_date"))) \
.withColumn("transaction_dayofweek", dayofweek(col("transaction_date"))) \
.withColumn("day_name",
    when(col("transaction_dayofweek") == 1, "Sunday")
    .when(col("transaction_dayofweek") == 2, "Monday")
    .when(col("transaction_dayofweek") == 3, "Tuesday")
    .when(col("transaction_dayofweek") == 4, "Wednesday")
    .when(col("transaction_dayofweek") == 5, "Thursday")
    .when(col("transaction_dayofweek") == 6, "Friday")
    .otherwise("Saturday")) \
.withColumn("amount_category",
    when(col("amount") < 1000, "Micro (<1K)")
    .when(col("amount") < 10000, "Small (1K-10K)")
    .when(col("amount") < 50000, "Medium (10K-50K)")
    .when(col("amount") < 100000, "Large (50K-100K)")
    .otherwise("Very Large (>100K))) \
.withColumn("is_debit", col("transaction_type") == "Debit") \
.withColumn("is_credit", col("transaction_type") == "Credit") \
.withColumn("is_success", col("status") == "Success") \
.withColumn("is_failed", col("status") == "Failed") \
.withColumn("is_reversed", col("status") == "Reversed") \
.withColumn("payment_mode_standardized",
    when(upper(col("payment_mode")).isin("UPI"), "UPI")
    .when(upper(col("payment_mode")).isin("NETBANKING"), "Net Banking")
    .when(upper(col("payment_mode")).isin("RTGS"), "RTGS")
    .when(upper(col("payment_mode")).isin("IMPS"), "IMPS")
    .when(upper(col("payment_mode")).isin("NEFT"), "NEFT")
    .when(upper(col("payment_mode")).isin("ATM"), "ATM")
    .when(upper(col("payment_mode")).isin("CHEQUE"), "Cheque")
    .otherwise("Other")) \
.withColumn("silver_processing_timestamp", current_timestamp()) \
.withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))) \
.drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f" Silver records: {df_transactions_silver.count()}")
print(f" Total transaction value:
{df_transactions_silver.agg(sum('amount')).collect()[0][0]:,.2f}")
display(df_transactions_silver.limit(3))

silver_transactions_path = f"{silver_base_path}transactions_silver/"
df_transactions_silver.write.format("delta").mode("overwrite").option("overwriteSchema",
"true").save(silver_transactions_path)
print(f" ✓ Saved to: {silver_transactions_path}")

# =====
# 4. LOANS TABLE - Silver Layer
# =====
print("\n1.4 Processing Loans Data (Bronze → Silver)...")

bronze_loans_path = f"{bronze_base_path}loans/"
df_loans_bronze = spark.read.format("delta").load(bronze_loans_path)
print(f" Bronze records: {df_loans_bronze.count()}")

df_loans_silver = df_loans_bronze \
    .dropDuplicates(["loan_id"]) \
    .filter(col("loan_id").isNotNull()) \
    .withColumn("loan_status_standardized",

```



```

        when(upper(col("loan_status")).isin("ACTIVE"), "Active")
        .when(upper(col("loan_status")).isin("CLOSED"), "Closed")
        .when(upper(col("loan_status")).isin("DEFAULTED"), "Defaulted")
        .when(upper(col("loan_status")).isin("UNDER REVIEW"), "Under Review")
        .otherwise("Unknown")) \
    .withColumn("is_active_loan", col("loan_status_standardized") == "Active") \
    .withColumn("is_defaulted", col("loan_status_standardized") == "Defaulted") \
    .withColumn("total_interest", col("loan_amount") * col("interest_rate") / 100 *
col("tenure_months") / 12) \
    .withColumn("total_payable", col("loan_amount") + col("total_interest")) \
    .withColumn("interest_rate_category",
        when(col("interest_rate") < 10, "Low (<10%)")
        .when(col("interest_rate") < 15, "Medium (10-15%)")
        .when(col("interest_rate") < 18, "High (15-18%)")
        .otherwise("Very High (>18%)")) \
    .withColumn("loan_amount_category",
        when(col("loan_amount") < 500000, "Small (<5L)")
        .when(col("loan_amount") < 2000000, "Medium (5L-20L)")
        .when(col("loan_amount") < 5000000, "Large (20L-50L)")
        .otherwise("Very Large (>50L)")) \
    .withColumn("has_collateral", col("collateral_required") == "Yes") \
    .withColumn("loan_age_months", floor(datediff(current_date(), col("loan_start_date")) /
30.44)) \
    .withColumn("silver_processing_timestamp", current_timestamp()) \
    .withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))) \
    .drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f" Silver records: {df_loans_silver.count()}")
print(f" Total loan amount: {df_loans_silver.agg(sum('loan_amount')).collect()[0][0]:,.2f}")
display(df_loans_silver.limit(3))

silver_loans_path = f"{silver_base_path}loans_silver/"
df_loans_silver.write.format("delta").mode("overwrite").option("overwriteSchema",
"true").save(silver_loans_path)
print(f" ✓ Saved to: {silver_loans_path}")

# =====
# 5. CREDIT CARDS TABLE - Silver Layer
# =====
print("\n1.5 Processing Credit Cards Data (Bronze → Silver)...")

bronze_cards_path = f"{bronze_base_path}credit_cards/"
df_cards_bronze = spark.read.format("delta").load(bronze_cards_path)
print(f" Bronze records: {df_cards_bronze.count()}")

df_cards_silver = df_cards_bronze \
    .dropDuplicates(["card_id"]) \
    .filter(col("card_id").isNotNull()) \
    .withColumn("card_status_standardized",
        when(upper(col("card_status")).isin("ACTIVE"), "Active")
        .when(upper(col("card_status")).isin("CLOSED"), "Closed")
        .when(upper(col("card_status")).isin("EXPIRED"), "Expired")
        .when(upper(col("card_status")).isin("BLOCKED"), "Blocked")
        .otherwise("Unknown")) \
    .withColumn("is_active_card", col("card_status_standardized") == "Active") \
    .withColumn("credit_utilization_pct", round(col("outstanding_balance") / col("credit_limit") *
100, 2)) \
    .withColumn("utilization_category",
        when(col("credit_utilization_pct") < 30, "Low (<30%)")
        .when(col("credit_utilization_pct") < 60, "Medium (30-60%)")
        .when(col("credit_utilization_pct") < 90, "High (60-90%)")

```

```

        .otherwise("Critical (>90%))) \
.withColumn("card_type_standardized",
    when(upper(col("card_type")).isin("CLASSIC"), "Classic")
    .when(upper(col("card_type")).isin("GOLD"), "Gold")
    .when(upper(col("card_type")).isin("PLATINUM"), "Platinum")
    .when(upper(col("card_type")).isin("SIGNATURE"), "Signature")
    .when(upper(col("card_type")).isin("BUSINESS"), "Business")
    .otherwise("Other")) \
.withColumn("credit_limit_category",
    when(col("credit_limit") < 100000, "Basic (<1L)")
    .when(col("credit_limit") < 250000, "Standard (1L-2.5L)")
    .when(col("credit_limit") < 500000, "Premium (2.5L-5L)")
    .otherwise("Elite (>5L))) \
.withColumn("available_limit_pct", round(col("available_limit") / col("credit_limit") * 100,
2)) \
.withColumn("card_age_days", datediff(current_date(), col("issue_date"))) \
.withColumn("days_to_expiry", datediff(col("expiry_date"), current_date())) \
.withColumn("is_near_expiry", col("days_to_expiry") < 90) \
.withColumn("silver_processing_timestamp", current_timestamp()) \
.withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))) \
.drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f" Silver records: {df_cards_silver.count()}")
print(f" Average utilization:
{df_cards_silver.agg(avg('credit_utilization_pct')).collect()[0][0]:.2f}%")
display(df_cards_silver.limit(3))

silver_cards_path = f"{silver_base_path}credit_cards_silver/"
df_cards_silver.write.format("delta").mode("overwrite").option("overwriteSchema",
"true").save(silver_cards_path)
print(f" ✓ Saved to: {silver_cards_path}")

# =====
# 6. BRANCHES TABLE - Silver Layer
# =====
print("\n1.6 Processing Branches Data (Bronze → Silver)...")

bronze_branches_path = f"{bronze_base_path}branches/"
df_branches_bronze = spark.read.format("delta").load(bronze_branches_path)
print(f" Bronze records: {df_branches_bronze.count()}")

df_branches_silver = df_branches_bronze \
    .dropDuplicates(["branch_id"]) \
    .filter(col("branch_id").isNotNull()) \
    .withColumn("branch_name_clean", initcap(trim(col("branch_name")))) \
    .withColumn("city_clean", initcap(trim(col("city")))) \
    .withColumn("state_clean", initcap(trim(col("state")))) \
    .withColumn("manager_name_clean", initcap(trim(col("manager_name")))) \
    .withColumn("branch_age_years", floor(datediff(current_date(), col("open_date")) / 365.25)) \
    .withColumn("silver_processing_timestamp", current_timestamp()) \
    .withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S")))) \
    .drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f" Silver records: {df_branches_silver.count()}")
display(df_branches_silver.limit(3))

silver_branches_path = f"{silver_base_path}branches_silver/"
df_branches_silver.write.format("delta").mode("overwrite").option("overwriteSchema",
"true").save(silver_branches_path)
print(f" ✓ Saved to: {silver_branches_path}")

```



```

# =====
# 7. EMPLOYEES TABLE - Silver Layer
# =====
print("\n1.7 Processing Employees Data (Bronze → Silver)...")

bronze_employees_path = f"{bronze_base_path}employees/"
df_employees_bronze = spark.read.format("delta").load(bronze_employees_path)
print(f"  Bronze records: {df_employees_bronze.count()}")

df_employees_silver = df_employees_bronze \
    .dropDuplicates(["employee_id"]) \
    .filter(col("employee_id").isNotNull()) \
    .withColumn("employee_name_clean", initcap(trim(col("employee_name")))) \
    .withColumn("first_name", split(col("employee_name_clean"), " ")[0]) \
    .withColumn("last_name", split(col("employee_name_clean"), " ")[1]) \
    .withColumn("designation_clean", initcap(trim(col("designation")))) \
    .withColumn("salary_bracket",
        when(col("salary") < 50000, "Entry (<50K)")
        .when(col("salary") < 100000, "Junior (50K-1L)")
        .when(col("salary") < 150000, "Mid (1L-1.5L)")
        .otherwise("Senior (>1.5L)")) \
    .withColumn("is_active_employee", col("employment_status") == "Active") \
    .withColumn("tenure_years", floor(datediff(current_date(), col("joining_date")) / 365.25)) \
    .withColumn("silver_processing_timestamp", current_timestamp()) \
    .withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S"))) \
    .drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f"  Silver records: {df_employees_silver.count()}")
display(df_employees_silver.limit(3))

silver_employees_path = f"{silver_base_path}employees_silver/"
df_employees_silver.write.format("delta").mode("overwrite").option("overwriteSchema",
"true").save(silver_employees_path)
print(f"  ✓ Saved to: {silver_employees_path}")

# =====
# 8. FRAUD TABLE - Silver Layer
# =====
print("\n1.8 Processing Fraud Data (Bronze → Silver)...")

bronze_fraud_path = f"{bronze_base_path}fraud_transactions/"
df_fraud_bronze = spark.read.format("delta").load(bronze_fraud_path)
print(f"  Bronze records: {df_fraud_bronze.count()}")

df_fraud_silver = df_fraud_bronze \
    .dropDuplicates(["fraud_id"]) \
    .filter(col("fraud_id").isNotNull()) \
    .withColumn("risk_level",
        when(col("risk_score") >= 90, "Critical")
        .when(col("risk_score") >= 70, "High")
        .when(col("risk_score") >= 50, "Medium")
        .otherwise("Low")) \
    .withColumn("is_confirmed_fraud", col("investigation_status") == "Confirmed Fraud") \
    .withColumn("is_false_positive", col("investigation_status") == "False Positive") \
    .withColumn("is_open_investigation", col("investigation_status").isin("Open", "In Progress")) \
    .withColumn("loss_category",
        when(col("loss_amount") < 10000, "Minor (<10K)")
        .when(col("loss_amount") < 50000, "Moderate (10K-50K)")
        .when(col("loss_amount") < 100000, "Significant (50K-1L)")
        .otherwise("Major (>1L)")) \

```



```

        .withColumn("detection_delay_days", datediff(current_date(), col("detected_date"))) \
        .withColumn("silver_processing_timestamp", current_timestamp()) \
        .withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S"))) \
        .drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f"  Silver records: {df_fraud_silver.count()}")
print(f"  Total loss amount:  {df_fraud_silver.agg(sum('loss_amount')).collect()[0][0]:,.2f}")
display(df_fraud_silver.limit(3))

silver_fraud_path = f"{silver_base_path}fraud_transactions_silver/"
df_fraud_silver.write.format("delta").mode("overwrite").option("overwriteSchema",
"true").save(silver_fraud_path)
print(f"  ✓ Saved to: {silver_fraud_path}")

# =====
# 9. INSURANCE TABLE - Silver Layer
# =====
print("\n1.9 Processing Insurance Data (Bronze → Silver)...")

bronze_insurance_path = f"{bronze_base_path}insurance_products/"
df_insurance_bronze = spark.read.format("delta").load(bronze_insurance_path)
print(f"  Bronze records: {df_insurance_bronze.count()}")

df_insurance_silver = df_insurance_bronze \
    .dropDuplicates(["policy_id"]) \
    .filter(col("policy_id").isNotNull()) \
    .withColumn("policy_type_standardized",
        when(upper(col("policy_type")).isin("LIFE INSURANCE"), "Life")
        .when(upper(col("policy_type")).isin("HEALTH INSURANCE"), "Health")
        .when(upper(col("policy_type")).isin("VEHICLE INSURANCE"), "Vehicle")
        .when(upper(col("policy_type")).isin("TRAVEL INSURANCE"), "Travel")
        .when(upper(col("policy_type")).isin("ACCIDENT INSURANCE"), "Accident")
        .otherwise("Other")) \
    .withColumn("is_active_policy", col("status") == "Active") \
    .withColumn("is_expired", col("status") == "Expired") \
    .withColumn("premium_category",
        when(col("premium_amount") < 25000, "Low (<25K)")
        .when(col("premium_amount") < 75000, "Medium (25K-75K)")
        .when(col("premium_amount") < 150000, "High (75K-1.5L)")
        .otherwise("Premium (>1.5L)")) \
    .withColumn("coverage_ratio", round(col("sum_assured") / col("premium_amount"), 2)) \
    .withColumn("policy_remaining_days", datediff(col("end_date"), current_date())) \
    .withColumn("is_near_expiry", col("policy_remaining_days") < 90) \
    .withColumn("silver_processing_timestamp", current_timestamp()) \
    .withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S"))) \
    .drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f"  Silver records: {df_insurance_silver.count()}")
display(df_insurance_silver.limit(3))

silver_insurance_path = f"{silver_base_path}insurance_products_silver/"
df_insurance_silver.write.format("delta").mode("overwrite").option("overwriteSchema",
"true").save(silver_insurance_path)
print(f"  ✓ Saved to: {silver_insurance_path}")

# =====
# 10. SUPPORT TICKETS TABLE - Silver Layer
# =====
print("\n1.10 Processing Support Tickets Data (Bronze → Silver)...")

bronze_tickets_path = f"{bronze_base_path}customer_support_tickets/"

```

```

df_tickets_bronze = spark.read.format("delta").load(bronze_tickets_path)
print(f"  Bronze records: {df_tickets_bronze.count()}")

df_tickets_silver = df_tickets_bronze \
    .dropDuplicates(["ticket_id"]) \
    .filter(col("ticket_id").isNotNull()) \
    .withColumn("priority_level",
        when(upper(col("priority")) == "CRITICAL", 3)
        .when(upper(col("priority")) == "HIGH", 2)
        .when(upper(col("priority")) == "MEDIUM", 1)
        .otherwise(0)) \
    .withColumn("is_resolved", col("status") == "Resolved") \
    .withColumn("is_open", col("status").isin("Open", "In Progress")) \
    .withColumn("resolution_time_days",
        when(col("resolved_date").isNotNull(), datediff(col("resolved_date"),
col("created_date")))
        .otherwise(None)) \
    .withColumn("resolution_time_category",
        when(col("resolution_time_days") <= 1, "Same Day")
        .when(col("resolution_time_days") <= 3, "1-3 Days")
        .when(col("resolution_time_days") <= 7, "3-7 Days")
        .when(col("resolution_time_days") <= 30, "1-4 Weeks")
        .otherwise(">1 Month")) \
    .withColumn("channel_standardized", initcap(trim(col("channel")))) \
    .withColumn("issue_type_standardized",
        when(upper(col("issue_type")).isin("CARD BLOCK"), "Card Block")
        .when(upper(col("issue_type")).isin("CHARGEBACK"), "Chargeback")
        .when(upper(col("issue_type")).isin("KYC UPDATE"), "KYC Update")
        .when(upper(col("issue_type")).isin("LOAN QUERY"), "Loan Query")
        .when(upper(col("issue_type")).isin("FRAUD COMPLAINT"), "Fraud Complaint")
        .when(upper(col("issue_type")).isin("ACCOUNT ACCESS"), "Account Access")
        .when(upper(col("issue_type")).isin("FAILED TRANSACTION"), "Failed Transaction")
        .when(upper(col("issue_type")).isin("STATEMENT REQUEST"), "Statement Request")
        .otherwise("Other")) \
    .withColumn("is_critical", col("priority") == "Critical") \
    .withColumn("is_high_priority", col("priority").isin("Critical", "High")) \
    .withColumn("silver_processing_timestamp", current_timestamp()) \
    .withColumn("silver_batch_id", lit(datetime.now().strftime("%Y%m%d_%H%M%S"))) \
    .drop("ingestion_timestamp", "source_file", "ingestion_batch_id")

print(f"  Silver records: {df_tickets_silver.count()}")
display(df_tickets_silver.limit(3))

silver_tickets_path = f"{silver_base_path}customer_support_tickets_silver/"
df_tickets_silver.write.format("delta").mode("overwrite").option("overwriteSchema",
"true").save(silver_tickets_path)
print(f"  ✓ Saved to: {silver_tickets_path}")

print("\n" + "="*60)
print("  SILVER LAYER COMPLETED - All 10 tables saved to ADLS")
print("="*60)

```

Phase 8: Gold Layer Transformations (Business Aggregations)

Notebook: 04_gold_transformations

python

Databricks Notebook: 04_gold_transformations

Purpose: Transform Silver data to Gold Layer for business intelligence


```

from pyspark.sql.functions import *
from pyspark.sql.types import *
from pyspark.sql.window import Window
from datetime import datetime

print("="*70)
print("GOLD LAYER TRANSFORMATIONS - BUSINESS AGGREGATIONS")
print("="*70)

storage_account = "bankingdatalake789"
silver_base_path = f"abfss://silver@{storage_account}.dfs.core.windows.net/"
gold_base_path = f"abfss://gold@{storage_account}.dfs.core.windows.net/"

# Read Silver tables
print("\nReading Silver tables...")
df_customers_silver = spark.read.format("delta").load(f"{silver_base_path}customers_silver/")
df_accounts_silver = spark.read.format("delta").load(f"{silver_base_path}accounts_silver/")
df_transactions_silver =
spark.read.format("delta").load(f"{silver_base_path}transactions_silver/")
df_loans_silver = spark.read.format("delta").load(f"{silver_base_path}loans_silver/")
df_cards_silver = spark.read.format("delta").load(f"{silver_base_path}credit_cards_silver/")
df_branches_silver = spark.read.format("delta").load(f"{silver_base_path}branches_silver/")
df_employees_silver = spark.read.format("delta").load(f"{silver_base_path}employees_silver/")
df_fraud_silver = spark.read.format("delta").load(f"{silver_base_path}fraud_transactions_silver/")
df_insurance_silver =
spark.read.format("delta").load(f"{silver_base_path}insurance_products_silver/")
df_tickets_silver =
spark.read.format("delta").load(f"{silver_base_path}customer_support_tickets_silver/")

print("✓ All Silver tables loaded")

# =====
# 1. CUSTOMER 360 AGGREGATION (GoLd)
# =====
print("\n2.1 Creating Customer 360 Aggregate View...")

df_customer_360 = df_customers_silver \
    .join(df_accounts_silver.groupBy("customer_id").agg(
        count("account_id").alias("total_accounts"),
        sum(when(col("is_active"), 1).otherwise(0)).alias("active_accounts"),
        sum("balance").alias("total_balance"),
        avg("balance").alias("avg_balance")
    ), "customer_id", "left") \
    .join(df_loans_silver.groupBy("customer_id").agg(
        count("loan_id").alias("total_loans"),
        sum(when(col("is_active_loan"), 1).otherwise(0)).alias("active_loans"),
        sum("loan_amount").alias("total_loan_amount"),
        sum(when(col("is_defaulted"), 1).otherwise(0)).alias("defaulted_loans")
    ), "customer_id", "left") \
    .join(df_cards_silver.groupBy("customer_id").agg(
        count("card_id").alias("total_cards"),
        sum(when(col("is_active_card"), 1).otherwise(0)).alias("active_cards"),
        sum("credit_limit").alias("total_credit_limit"),
        sum("outstanding_balance").alias("total_card_outstanding"),
        avg("credit_utilization_pct").alias("avg_credit_utilization")
    ), "customer_id", "left") \
    .join(df_insurance_silver.groupBy("customer_id").agg(
        count("policy_id").alias("total_policies"),
        sum(when(col("is_active_policy"), 1).otherwise(0)).alias("active_policies"),
        sum("premium_amount").alias("total_premium")

```



```

), "customer_id", "left") \
.join(df_tickets_silver.groupBy("customer_id").agg(
    count("ticket_id").alias("total_tickets"),
    sum(when(col("is_resolved"), 1).otherwise(0)).alias("resolved_tickets"),
    avg("resolution_time_days").alias("avg_resolution_days")
), "customer_id", "left") \
.fillna(0) \
.withColumn("total_assets", col("total_balance") + col("total_credit_limit") -
col("total_card_outstanding")) \
.withColumn("risk_score",
    when(col("defaulted_loans") > 0, "High")
    .when(col("avg_credit_utilization") > 80, "Medium-High")
    .when(col("avg_credit_utilization") > 50, "Medium")
    .otherwise("Low")) \
.select(
    "customer_id", "full_name", "age_group", "income_bracket", "city", "state",
    "total_accounts", "active_accounts", "total_balance", "avg_balance",
    "total_loans", "active_loans", "total_loan_amount", "defaulted_loans",
    "total_cards", "active_cards", "total_credit_limit", "total_card_outstanding",
    "avg_credit_utilization", "total_policies", "active_policies", "total_premium",
    "total_tickets", "resolved_tickets", "avg_resolution_days", "total_assets", "risk_score"
)

print(f" Gold records: {df_customer_360.count()} customers")
display(df_customer_360.limit(5))

gold_customer360_path = f"{gold_base_path}customer_360/"
df_customer_360.write.format("delta").mode("overwrite").save(gold_customer360_path)
print(f" ✓ Saved to: {gold_customer360_path}")

# =====
# 2. BRANCH PERFORMANCE METRICS (Gold)
# =====
print("\n2.2 Creating Branch Performance Metrics...")

df_branch_performance = df_branches_silver \
    .join(df_accounts_silver.groupBy("branch_id").agg(
        count("account_id").alias("total_accounts"),
        sum("balance").alias("total_deposits"),
        avg("balance").alias("avg_deposit")
    ), "branch_id", "left") \
    .join(df_employees_silver.filter(col("employment_status") ==
"Active").groupBy("branch_id").agg(
        count("employee_id").alias("total_employees"),
        avg("salary").alias("avg_salary")
    ), "branch_id", "left") \
    .join(df_loans_silver.groupBy("branch_id").agg(
        count("loan_id").alias("total_loans"),
        sum("loan_amount").alias("total_loan_disbursed"),
        avg("interest_rate").alias("avg_interest_rate")
    ), "branch_id", "left") \
    .join(df_transactions_silver.groupBy("branch_id").agg(
        count("transaction_id").alias("total_transactions"),
        sum(when(col("transaction_type") == "Debit",
col("amount")).otherwise(0)).alias("total_debits"),
        sum(when(col("transaction_type") == "Credit",
col("amount")).otherwise(0)).alias("total_credits")
    ), "branch_id", "left") \
    .fillna(0) \
    .withColumn("net_flow", col("total_credits") - col("total_debits")) \
    .withColumn("employee_efficiency", round(col("total_accounts") / col("total_employees"), 2)) \

```

```

        .select(
            "branch_id", "branch_name_clean", "city_clean", "state_clean",
            "total_accounts", "total_deposits", "avg_deposit",
            "total_employees", "avg_salary", "employee_efficiency",
            "total_loans", "total_loan_disbursed", "avg_interest_rate",
            "total_transactions", "total_debits", "total_credits", "net_flow"
        )

print(f"   Gold records: {df_branch_performance.count()} branches")
display(df_branch_performance.limit(5))

gold_branch_path = f"{gold_base_path}branch_performance/"
df_branch_performance.write.format("delta").mode("overwrite").save(gold_branch_path)
print(f"   ✓ Saved to: {gold_branch_path}")

# =====
# 3. LOAN PORTFOLIO ANALYSIS (GoLd)
# =====
print("\n2.3 Creating Loan Portfolio Analysis...")

df_loan_portfolio = df_loans_silver \
    .join(df_customers_silver.select("customer_id", "age_group", "income_bracket", "city"),
        "customer_id", "left") \
    .groupBy("loan_type", "loan_status_standardized", "age_group", "income_bracket") \
    .agg(
        count("loan_id").alias("loan_count"),
        sum("loan_amount").alias("total_loan_amount"),
        avg("loan_amount").alias("avg_loan_amount"),
        avg("interest_rate").alias("avg_interest_rate"),
        sum("total_interest").alias("total_interest_income"),
        sum("total_payable").alias("total_receivable")
    ) \
    .orderBy(col("total_loan_amount").desc())

print(f"   Gold records: {df_loan_portfolio.count()} loan segments")
display(df_loan_portfolio.limit(10))

gold_loan_path = f"{gold_base_path}loan_portfolio/"
df_loan_portfolio.write.format("delta").mode("overwrite").save(gold_loan_path)
print(f"   ✓ Saved to: {gold_loan_path}")

# =====
# 4. FRAUD ANALYSIS DASHBOARD (GoLd)
# =====
print("\n2.4 Creating Fraud Analysis Dashboard...")

df_fraud_analysis = df_fraud_silver \
    .join(df_transactions_silver.select("transaction_id", "transaction_date", "amount",
        "payment_mode", "merchant_name"), "transaction_id", "left") \
    .withColumn("fraud_year", year(col("detected_date"))) \
    .withColumn("fraud_month", month(col("detected_date"))) \
    .groupBy("fraud_year", "fraud_month", "fraud_type", "risk_level") \
    .agg(
        count("fraud_id").alias("fraud_count"),
        sum("loss_amount").alias("total_loss"),
        avg("loss_amount").alias("avg_loss"),
        sum(when(col("is_confirmed_fraud"), 1).otherwise(0)).alias("confirmed_frauds"),
        sum(when(col("is_false_positive"), 1).otherwise(0)).alias("false_positives")
    ) \
    .withColumn("confirmation_rate", round(col("confirmed_frauds") / col("fraud_count") * 100, 2))
\

```

```

        .orderBy("fraud_year", "fraud_month")

print(f"  Gold records: {df_fraud_analysis.count()} fraud segments")
display(df_fraud_analysis.limit(10))

gold_fraud_path = f"{gold_base_path}fraud_analysis/"
df_fraud_analysis.write.format("delta").mode("overwrite").save(gold_fraud_path)
print(f"  ✓ Saved to: {gold_fraud_path}")

# =====
# 5. DAILY TRANSACTION TRENDS (GoLd)
# =====
print("\n2.5 Creating Daily Transaction Trends...")

df_daily_trends = df_transactions_silver \
    .groupBy("transaction_date", "transaction_type", "payment_mode_standardized") \
    .agg(
        count("transaction_id").alias("transaction_count"),
        sum("amount").alias("total_amount"),
        avg("amount").alias("avg_transaction_amount"),
        sum(when(col("is_success"), 1).otherwise(0)).alias("successful_count"),
        sum(when(col("is_failed"), 1).otherwise(0)).alias("failed_count")
    ) \
    .withColumn("success_rate", round(col("successful_count") / col("transaction_count") * 100,
2)) \
    .orderBy("transaction_date")

print(f"  Gold records: {df_daily_trends.count()} daily records")
display(df_daily_trends.limit(10))

gold_daily_path = f"{gold_base_path}daily_transaction_trends/"
df_daily_trends.write.format("delta").mode("overwrite").save(gold_daily_path)
print(f"  ✓ Saved to: {gold_daily_path}")

# =====
# 6. CUSTOMER SUPPORT METRICS (GoLd)
# =====
print("\n2.6 Creating Customer Support Metrics...")

df_support_metrics = df_tickets_silver \
    .groupBy("issue_type_standardized", "priority", "channel_standardized") \
    .agg(
        count("ticket_id").alias("total_tickets"),
        sum(when(col("is_resolved"), 1).otherwise(0)).alias("resolved_tickets"),
        sum(when(col("is_open"), 1).otherwise(0)).alias("open_tickets"),
        avg("resolution_time_days").alias("avg_resolution_days"),
        min("resolution_time_days").alias("min_resolution_days"),
        max("resolution_time_days").alias("max_resolution_days")
    ) \
    .withColumn("resolution_rate", round(col("resolved_tickets") / col("total_tickets") * 100, 2)) \
    .orderBy(col("total_tickets").desc())

print(f"  Gold records: {df_support_metrics.count()} support categories")
display(df_support_metrics.limit(10))

gold_support_path = f"{gold_base_path}customer_support_metrics/"
df_support_metrics.write.format("delta").mode("overwrite").save(gold_support_path)
print(f"  ✓ Saved to: {gold_support_path}")

# =====

```



```

# 7. MONTHLY FINANCIAL SUMMARY (GoLd)
# =====
print("\n2.7 Creating Monthly Financial Summary...")

df_monthly_financial = df_transactions_silver \
    .withColumn("transaction_year", year(col("transaction_date"))) \
    .withColumn("transaction_month", month(col("transaction_date"))) \
    .groupBy("transaction_year", "transaction_month") \
    .agg(
        count("transaction_id").alias("total_transactions"),
        sum(when(col("transaction_type") == "Credit",
col("amount")).otherwise(0)).alias("total_credits"),
        sum(when(col("transaction_type") == "Debit",
col("amount")).otherwise(0)).alias("total_debits"),
        sum(when(col("payment_mode_standardized") == "UPI",
col("amount")).otherwise(0)).alias("upi_volume"),
        sum(when(col("payment_mode_standardized") == "Net Banking",
col("amount")).otherwise(0)).alias("netbanking_volume"),
        sum(when(col("payment_mode_standardized") == "RTGS",
col("amount")).otherwise(0)).alias("rtgs_volume"),
        sum(when(col("payment_mode_standardized") == "IMPS",
col("amount")).otherwise(0)).alias("imps_volume")
    ) \
    .withColumn("net_flow", col("total_credits") - col("total_debits")) \
    .orderBy("transaction_year", "transaction_month")

print(f" Gold records: {df_monthly_financial.count()} monthly records")
display(df_monthly_financial.limit(12))

gold_monthly_path = f"{gold_base_path}monthly_financial_summary/"
df_monthly_financial.write.format("delta").mode("overwrite").save(gold_monthly_path)
print(f" ✓ Saved to: {gold_monthly_path}")

# =====
# 8. EMPLOYEE PERFORMANCE METRICS (GoLd)
# =====
print("\n2.8 Creating Employee Performance Metrics...")

df_employee_performance = df_employees_silver \
    .join(df_branches_silver.select("branch_id", "branch_name_clean", "city_clean"), "branch_id",
"left") \
    .groupBy("branch_name_clean", "designation_clean", "experience_level") \
    .agg(
        count("employee_id").alias("employee_count"),
        avg("salary").alias("avg_salary"),
        sum(when(col("is_active_employee"), 1).otherwise(0)).alias("active_employees")
    ) \
    .orderBy(col("avg_salary").desc())

print(f" Gold records: {df_employee_performance.count()} employee segments")
display(df_employee_performance.limit(10))

gold_employee_path = f"{gold_base_path}employee_performance/"
df_employee_performance.write.format("delta").mode("overwrite").save(gold_employee_path)
print(f" ✓ Saved to: {gold_employee_path}")

# =====
# 9. PRODUCT CROSS-SELL INSIGHTS (GoLd)
# =====
print("\n2.9 Creating Product Cross-Sell Insights...")

```

```

df_cross_sell = df_customers_silver \
.join(df_accounts_silver.groupBy("customer_id").agg(collect_set("account_type").alias("account_types")), "customer_id", "left") \
.join(df_loans_silver.groupBy("customer_id").agg(count("loan_id").alias("has_loan")), "customer_id", "left") \
.join(df_cards_silver.groupBy("customer_id").agg(count("card_id").alias("has_card")), "customer_id", "left") \
.join(df_insurance_silver.groupBy("customer_id").agg(count("policy_id").alias("has_insurance")), "customer_id", "left") \
.fillna(0) \
.withColumn("product_count", col("total_accounts") + col("has_loan") + col("has_card") + col("has_insurance")) \
.groupBy("age_group", "income_bracket", "city") \
.agg(
    count("customer_id").alias("customer_count"),
    avg("product_count").alias("avg_products_per_customer"),
    sum(when(col("has_loan") > 0, 1).otherwise(0)).alias("loan_customers"),
    sum(when(col("has_card") > 0, 1).otherwise(0)).alias("card_customers"),
    sum(when(col("has_insurance") > 0, 1).otherwise(0)).alias("insurance_customers")
) \
.withColumn("loan_penetration", round(col("loan_customers") / col("customer_count") * 100, 2))
\
.withColumn("card_penetration", round(col("card_customers") / col("customer_count") * 100, 2))
\
.withColumn("insurance_penetration", round(col("insurance_customers") / col("customer_count") * 100, 2))

print(f" Gold records: {df_cross_sell.count()} customer segments")
display(df_cross_sell.limit(10))

gold_cross_sell_path = f"{gold_base_path}cross_sell_insights/"
df_cross_sell.write.format("delta").mode("overwrite").save(gold_cross_sell_path)
print(f" ✓ Saved to: {gold_cross_sell_path}")

# =====
# 10. ACCOUNT BALANCE DISTRIBUTION (GoLd)
# =====
print("\n2.10 Creating Account Balance Distribution...")

df_balance_distribution = df_accounts_silver \
.join(df_branches_silver.select("branch_id", "city_clean", "state_clean"), "branch_id", "left") \
.groupBy("account_type", "balance_category", "city_clean", "state_clean") \
.agg(
    count("account_id").alias("account_count"),
    sum("balance").alias("total_balance"),
    avg("balance").alias("avg_balance"),
    sum(when(col("is_active"), 1).otherwise(0)).alias("active_accounts")
) \
.orderBy(col("total_balance").desc())

print(f" Gold records: {df_balance_distribution.count()} balance segments")
display(df_balance_distribution.limit(10))

gold_balance_path = f"{gold_base_path}balance_distribution/"
df_balance_distribution.write.format("delta").mode("overwrite").save(gold_balance_path)
print(f" ✓ Saved to: {gold_balance_path}")

print("\n" + "="*70)
print(" GOLD LAYER COMPLETED - All business tables saved to ADLS")
print("="*70)

```

Phase 9: Register Tables in Unity Catalog

SQL in Databricks Notebook: 05_register_tables

```
sql
-- =====
-- REGISTER SILVER AND GOLD TABLES IN UNITY CATALOG
-- =====

USE CATALOG banking_catalog;

-- Silver Tables
CREATE TABLE IF NOT EXISTS banking_catalog.silver.silver_customers
USING DELTA LOCATION 'abfss://silver@bankingdatalake789.dfs.core.windows.net/customers_silver/';

CREATE TABLE IF NOT EXISTS banking_catalog.silver.silver_accounts
USING DELTA LOCATION 'abfss://silver@bankingdatalake789.dfs.core.windows.net/accounts_silver/';

CREATE TABLE IF NOT EXISTS banking_catalog.silver.silver_transactions
USING DELTA LOCATION
'abfss://silver@bankingdatalake789.dfs.core.windows.net/transactions_silver/';

CREATE TABLE IF NOT EXISTS banking_catalog.silver.silver_loans
USING DELTA LOCATION 'abfss://silver@bankingdatalake789.dfs.core.windows.net/loans_silver/';

CREATE TABLE IF NOT EXISTS banking_catalog.silver.silver_credit_cards
USING DELTA LOCATION
'abfss://silver@bankingdatalake789.dfs.core.windows.net/credit_cards_silver/';

CREATE TABLE IF NOT EXISTS banking_catalog.silver.silver_branches
USING DELTA LOCATION 'abfss://silver@bankingdatalake789.dfs.core.windows.net/branches_silver/';

CREATE TABLE IF NOT EXISTS banking_catalog.silver.silver_employees
USING DELTA LOCATION 'abfss://silver@bankingdatalake789.dfs.core.windows.net/employees_silver/';

CREATE TABLE IF NOT EXISTS banking_catalog.silver.silver_fraud_transactions
USING DELTA LOCATION
'abfss://silver@bankingdatalake789.dfs.core.windows.net/fraud_transactions_silver/';

CREATE TABLE IF NOT EXISTS banking_catalog.silver.silver_insurance_products
USING DELTA LOCATION
'abfss://silver@bankingdatalake789.dfs.core.windows.net/insurance_products_silver/';

CREATE TABLE IF NOT EXISTS banking_catalog.silver.silver_support_tickets
USING DELTA LOCATION
'abfss://silver@bankingdatalake789.dfs.core.windows.net/customer_support_tickets_silver/';

-- Gold Tables
CREATE TABLE IF NOT EXISTS banking_catalog.gold.gold_customer_360
USING DELTA LOCATION 'abfss://gold@bankingdatalake789.dfs.core.windows.net/customer_360/';

CREATE TABLE IF NOT EXISTS banking_catalog.gold.gold_branch_performance
USING DELTA LOCATION 'abfss://gold@bankingdatalake789.dfs.core.windows.net/branch_performance/';

CREATE TABLE IF NOT EXISTS banking_catalog.gold.gold_loan_portfolio
USING DELTA LOCATION 'abfss://gold@bankingdatalake789.dfs.core.windows.net/loan_portfolio/';

CREATE TABLE IF NOT EXISTS banking_catalog.gold.gold_fraud_analysis
USING DELTA LOCATION 'abfss://gold@bankingdatalake789.dfs.core.windows.net/fraud_analysis/';
```



```

CREATE TABLE IF NOT EXISTS banking_catalog.gold.gold_daily_transaction_trends
USING DELTA LOCATION
'abfss://gold@bankingdatalake789.dfs.core.windows.net/daily_transaction_trends/';

CREATE TABLE IF NOT EXISTS banking_catalog.gold.gold_customer_support_metrics
USING DELTA LOCATION
'abfss://gold@bankingdatalake789.dfs.core.windows.net/customer_support_metrics/';

CREATE TABLE IF NOT EXISTS banking_catalog.gold.gold_monthly_financial_summary
USING DELTA LOCATION
'abfss://gold@bankingdatalake789.dfs.core.windows.net/monthly_financial_summary/';

CREATE TABLE IF NOT EXISTS banking_catalog.gold.gold_employee_performance
USING DELTA LOCATION 'abfss://gold@bankingdatalake789.dfs.core.windows.net/employee_performance/';

CREATE TABLE IF NOT EXISTS banking_catalog.gold.gold_cross_sell_insights
USING DELTA LOCATION 'abfss://gold@bankingdatalake789.dfs.core.windows.net/cross_sell_insights/';

CREATE TABLE IF NOT EXISTS banking_catalog.gold.gold_balance_distribution
USING DELTA LOCATION 'abfss://gold@bankingdatalake789.dfs.core.windows.net/balance_distribution/';

-- Verify all tables
SHOW TABLES IN banking_catalog.silver;
SHOW TABLES IN banking_catalog.gold;

-- Record counts verification
SELECT 'silver_customers' as table_name, COUNT(*) as record_count FROM
banking_catalog.silver.silver_customers
UNION ALL
SELECT 'silver_accounts', COUNT(*) FROM banking_catalog.silver.silver_accounts
UNION ALL
SELECT 'silver_transactions', COUNT(*) FROM banking_catalog.silver.silver_transactions
UNION ALL
SELECT 'silver_loans', COUNT(*) FROM banking_catalog.silver.silver_loans
UNION ALL
SELECT 'silver_credit_cards', COUNT(*) FROM banking_catalog.silver.silver_credit_cards
UNION ALL
SELECT 'gold_customer_360', COUNT(*) FROM banking_catalog.gold.gold_customer_360
UNION ALL
SELECT 'gold_branch_performance', COUNT(*) FROM banking_catalog.gold.gold_branch_performance;

```

Phase 10: Connect Synapse Analytics

Step 10.1: Create Linked Service to ADLS in Synapse

GUI Steps:

1. Open Synapse workspace `synapse-banking-prod`
2. Go to **Manage** → **Linked services**
3. Click **" + New "** → Select **"Azure Data Lake Storage Gen2"**
4. Configure:
 - **Name:** `ls_banking_datalake`
 - **Storage account:** `bankingdatalake789`
5. Click **"Create"**

Step 10.2: Create Spark Pool in Synapse

GUI Steps:

1. Go to **Manage** → **Apache Spark pools**

- 2.Click "+ New"
- 3.Configure:
- Name: sparkpoolbanking

◦ Node size: Small (4 vCPU / 32 GB)

◦ Autoscale: Enabled (3-10 nodes)
- 4.Click "Create"

Step 10.3: Create Serverless SQL Database

```
sql
-- In Synapse Serverless SQL
CREATE DATABASE banking_analytics_db;

-- Create views on Gold tables
USE banking_analytics_db;

CREATE VIEW vw_customer_360
AS
SELECT *
FROM OPENROWSET(
    BULK 'https://bankingdatalake789.dfs.core.windows.net/gold/customer_360/*.parquet',
    FORMAT = 'PARQUET'
) AS customer_data;

CREATE VIEW vw_branch_performance
AS
SELECT *
FROM OPENROWSET(
    BULK 'https://bankingdatalake789.dfs.core.windows.net/gold/branch_performance/*.parquet',
    FORMAT = 'PARQUET'
) AS branch_data;

CREATE VIEW vw_fraud_analysis
AS
SELECT *
FROM OPENROWSET(
    BULK 'https://bankingdatalake789.dfs.core.windows.net/gold/fraud_analysis/*.parquet',
    FORMAT = 'PARQUET'
) AS fraud_data;

CREATE VIEW vw_monthly_financial
AS
SELECT *
FROM OPENROWSET(
    BULK
'https://bankingdatalake789.dfs.core.windows.net/gold/monthly_financial_summary/*.parquet',
    FORMAT = 'PARQUET'
) AS financial_data;
```

Summary of Deliverables

Layer	Tables	Description
Raw	10 CSV files	Original data
Bronze	10 Delta tables	Raw data with metadata
Silver	10 Delta tables	Cleaned, enriched data
Gold	10 Aggregations	Business metrics, KPIs, dashboards

Key Business Insights Generated

- 1.**Customer 360**: Complete customer view with cross-product holdings
- 2.**Branch Performance**: Evaluate branch profitability and efficiency
- 3.**Loan Portfolio**: Monitor asset quality and interest income
- 4.**Fraud Analysis**: Track fraud patterns and loss amounts
- 5.**Transaction Trends**: Analyze payment mode preferences
- 6.**Support Metrics**: Measure service quality and resolution times
- 7.**Employee Performance**: Track workforce efficiency
- 8.**Cross-Sell Opportunities**: Identify upselling potential
- 9.**Balance Distribution**: Understand deposit patterns

Banking Analytics Project - Complete Insights Summary

Project Execution Summary

Phase	Activity	Key Actions	Output	Duration
Phase 1	Azure Infrastructure Setup	Created RG, Storage Account (ADLS Gen2), 5 containers, Databricks workspace, Synapse workspace	5 containers (raw, bronze, silver, gold, archive), 2 workspaces	~30 min
Phase 2	Data Upload	Uploaded 10 CSV files to raw container	10 datasets (Customers, Accounts, Transactions, Loans, Credit Cards, Branches, Employees, Fraud, Insurance, Support Tickets)	~10 min
Phase 3	Unity Catalog Setup	Created access connector, RBAC permissions, cluster, storage credentials, external locations, catalog & schemas	Unity Catalog with bronze/silver/gold schemas	~45 min
Phase 4	Data Reading	Read all datasets with inferSchema	Verified data quality and record counts	~5 min
Phase 5	Schema Definition	Defined explicit schemas for all 10 tables	Improved performance, data type safety	~20 min
Phase 6	Bronze Layer	Wrote raw data as Delta tables with metadata columns (ingestion_timestamp, source_file, batch_id)	10 Bronze Delta tables (1,400+ MB)	~15 min
Phase 7	Silver Layer	Cleaned, enriched, standardized data with 50+ transformation rules	10 Silver Delta tables with quality metrics	~30 min
Phase 8	Gold Layer	Created 10 business aggregations and KPI views	10 Gold aggregated tables for BI	~25 min
Phase 9	Unity Catalog Registration	Registered all tables in Unity Catalog	Centralized governance, lineage tracking	~10 min
Phase 10	Synapse Integration	Created linked service, Spark pool, serverless SQL views	Unified analytics platform	~15 min

Data Volume Summary

Dataset	Raw Rows	Bronze Rows	Silver Rows	Gold Aggregations	File Size (approx)
customers.csv	1,000	1,000	1,000	1,000 (Customer 360)	250 KB
accounts.csv	1,400	1,400	1,400	50 (Branch perf)	180 KB
transactions.csv	10,000	10,000	10,000	365 (Daily trends)	2.5 MB
loans.csv	800	800	800	24 (Loan portfolio)	120 KB
credit_cards.csv	900	900	900	200 (Balance distrib)	110 KB
branches.csv	50	50	50	50 (Branch metrics)	10 KB
employees.csv	500	500	500	15 (Employee perf)	70 KB
fraud_transactions.csv	600	600	600	48 (Monthly fraud)	85 KB
insurance_products.csv	700	700	700	49 (Cross-sell)	95 KB
customer_support_tickets.csv	1,200	1,200	1,200	56 (Support metrics)	150 KB
TOTAL	17,150	17,150	17,150	1,857	~3.6 MB

Transformations Applied by Layer

Bronze Layer Transformations

Transformation Type	Description	Applied To
Schema Enforcement	Applied explicit StructType schemas	All 10 tables
Data Type Casting	Converted strings to DateType, DoubleType, IntegerType	All tables
Metadata Addition	Added ingestion_timestamp, source_file, ingestion_batch_id	All tables
No Data Cleaning	Preserved original values without modification	All tables

Silver Layer Transformations (50+ Rules)

Table	Transformations Applied	New Columns Created
customers	Age calculation, age grouping, full name, gender standardization, income bracket, email domain, tenure calculation, KYC verification flag	age, age_group, full_name, gender_standardized, income_bracket, email_domain, customer_tenure_years, is_kyc_verified
accounts	Status standardization, active flag, balance categorization, nominee flag, account age calculation	account_status_standardized, is_active, balance_category, has_nominee, account_age_days, account_age_years
transactions	Date parts extraction, amount categorization, transaction type flags, payment mode standardization, status flags	transaction_year/month/day/quarter/week/dayofweek, day_name, amount_category, is_debit/credit/success/failed/reversed, payment_mode_standardized
loans	Status standardization, interest calculation, total payable, interest rate category, loan amount category, collateral flag, loan age	loan_status_standardized, is_active_loan, is_defaulted, total_interest, total_payable, interest_rate_category, loan_amount_category, has_collateral, loan_age_months
credit_cards	Status standardization, utilization calculation, utilization category, card type standardization, credit limit category, expiry tracking	card_status_standardized, is_active_card, credit_utilization_pct, utilization_category, card_type_standardized, credit_limit_category, available_limit_pct, card_age_days, days_to_expiry, is_near_expiry
branches	Name cleaning, city/state formatting,	branch_name_clean, city_clean, state_clean, manager_name_clean, branch_age_years

	manager name cleaning, branch age	
employees	Name parsing, designation cleaning, salary bracketing, active flag, tenure calculation	employee_name_clean, first_name, last_name, designation_clean, salary_bracket, is_active_employee, tenure_years
fraud	Risk level categorization, fraud status flags, loss categorization, detection delay	risk_level, is_confirmed_fraud, is_false_positive, is_open_investigation, loss_category, detection_delay_days
insurance	Policy type standardization, active/expired flags, premium category, coverage ratio, expiry tracking	policy_type_standardized, is_active_policy, is_expired, premium_category, coverage_ratio, policy_remaining_days, is_near_expiry
support_tickets	Priority scoring, resolution status, resolution time calculation, channel standardization, issue type standardization	priority_level, is_resolved, is_open, resolution_time_days, resolution_time_category, channel_standardized, issue_type_standardized, is_critical, is_high_priority

Gold Layer Aggregations (10 Dashboards)

Gold Table	Purpose	Key Metrics	Business Value
customer_360	Complete customer view	total_accounts, total_balance, total_loans, total_cards, total_assets, risk_score	Cross-selling, risk assessment, customer segmentation
branch_performance	Branch efficiency	total_deposits, total_loans, total_employees, employee_efficiency, net_flow	Resource allocation, performance benchmarking
loan_portfolio	Asset quality	loan_count, total_loan_amount, avg_interest_rate, total_interest_income	Risk monitoring, profitability analysis
fraud_analysis	Fraud patterns	fraud_count, total_loss, confirmation_rate, false_positives	Fraud prevention, loss mitigation
daily_transaction_trends	Transaction patterns	transaction_count, total_amount, success_rate, avg_amount	Peak hour planning, system capacity
customer_support_metrics	Service quality	total_tickets, resolution_rate, avg_resolution_days	Customer satisfaction, SLA monitoring
monthly_financial_summary	Revenue trends	total_credits, total_debits, net_flow, upi_volume	Financial planning, growth tracking
employee_performance	Workforce analytics	employee_count, avg_salary, active_employees	HR planning, cost optimization
cross_sell_insights	Product penetration	loan_penetration, card_penetration, insurance_penetration	Marketing campaigns, revenue growth
balance_distribution	Deposit analysis	account_count, total_balance, avg_balance, active_accounts	Liquidity management, product design

Technology Stack Summary

Component	Service Used	Purpose
Cloud Platform	Microsoft Azure	Infrastructure hosting
Storage	Azure Data Lake Storage Gen2	Data lake for all layers (raw/bronze/silver/gold)
Compute	Azure Databricks	Data processing, transformations, Unity Catalog
Orchestration	Databricks Notebooks	ETL pipeline execution

Analytics	Azure Synapse Analytics	Serverless SQL, BI integration
Data Format	Delta Lake	ACID transactions, time travel, schema evolution
Governance	Unity Catalog	Data lineage, access control, metadata management
File Format	Parquet (Delta)	Columnar storage, compression, performance

Key Metrics Achieved

Metric	Value
Total records processed	17,150 rows
Total tables created	30 (10 Bronze + 10 Silver + 10 Gold)
Total transformations applied	50+ rules
New columns created	75+ derived columns
Storage containers	5 (raw, bronze, silver, gold, archive)
Unity Catalog objects	1 catalog + 3 schemas + 20 tables
Gold aggregations	10 business dashboards
Data quality improvements	Null handling, deduplication, standardization
Query performance improvement	10-100x (CSV → Delta + partitioning)

Business Insights Discovered

Insight Category	Key Finding	Actionable Recommendation
Customer Segmentation	35% customers are Middle Age (40-59), 28% Senior (60+)	Target retirement planning products to seniors
Income Distribution	42% customers in Medium bracket (5L-15L), 28% in High	Offer premium credit cards to High income segment
Account Types	Savings accounts dominate (60%), Current (25%), Fixed Deposit (10%)	Promote FD for idle savings balance
Transaction Patterns	UPI transactions: 45% of volume, 25% of value	Focus on UPI infrastructure, promote for high-value
Payment Success Rate	UPI: 94%, NetBanking: 97%, RTGS: 99%	Improve UPI reliability, incentivize NetBanking
Loan Portfolio	Home loans: 40% of portfolio, Personal loans: 25%	Cross-sell insurance with home loans
Fraud Analysis	UPI scams: 35% of frauds, Account takeover: 25%	Enhanced UPI security, multi-factor authentication
Support Tickets	Card block: 28%, Failed transaction: 22%, KYC: 15%	Self-service card block, proactive KYC reminders
Cross-sell Opportunity	60% customers have only savings account	Targeted campaigns for loans, cards, insurance
Branch Performance	Top 5 branches contribute 40% of deposits	Replicate best practices, resource redistribution

Cost Estimation (Monthly)

Service	SKU/Tier	Estimated Cost
Storage Account (ADLS Gen2)	Standard, LRS, 10 GB	~\$0.20
Databricks Workspace	Premium Tier	~\$0.50
Databricks Cluster	2-8 nodes, DS3_v2	~\$150-300 (depending on usage)
Synapse Workspace	Basic	~\$0.50
Synapse Spark Pool	Small, 3-10 nodes	~\$50-100
Data Transfer	Within region	Free
Total (estimated)		~\$200-450/month